



ECHOCeM: THIRD INTERFACE ECHO SEGMENTATION FOR CEMENT QUALITY ASSESSMENT

DATAChallenge DE DEEP

Auteurs: Víctor Camps Abarca
Aldric Bahna

Professeurs: Bertrand Michel
Didier Lime

NANTES, 16 JANVIER 2026

Table des matières

1	Introduction	1
1.1	Contexte	1
1.2	Outils utilisés	1
1.3	Métrique de performance	1
2	Exploration des données	1
2.1	Format des données	1
2.2	Distribution et déséquilibre des données	1
2.3	Pipeline de prétraitement et justification des techniques	2
3	Construction du modèle de référence	3
3.1	Motivations	3
3.2	Architecture générale	3
3.2.1	Structure du projet	3
3.2.2	Description des modules	4
3.2.3	Modèle UNet Baseline	4
3.2.4	Modifications ultérieures	5
4	Résultats	6
4.1	Métriques principales	6
4.2	Evolution des différentes métriques	7
4.3	Comparatif des métriques entre modèles	8
5	Conclusion	8
5.1	Résumé des travaux réalisés	8
5.2	Conclusion et résultats	9

Table des figures

1	Distribution des classes dans le dataset	2
2	Pipeline complet de prétraitement : Original $\rightarrow$ Redimensionné $\rightarrow$ Normalisé . .	2
3	Pipeline complet : entraînement (gauche) et inférence (droite)	3
4	Comparaison des IoU entre tous les modèles	7
5	Comparaison des Loss entre tous les modèles	7
6	Comparatif des métriques entre tous les modèles	8

Liste des tableaux

1	Comparaison des métriques clés	6
---	--	---

1 Introduction

1.1 Contexte

Ce travail porte sur l'analyse d'images ultrasonores issues d'inspections de puits, offrant des coupes radiales du complexe tubage-ciment-formation. Ces données, caractérisées par une forte variabilité et un bruit important, sont cruciales pour évaluer l'intégrité du ciment, garante de la sécurité des ouvrages. L'enjeu technique réside dans l'identification automatique, à l'échelle du pixel, des interfaces acoustiques, et particulièrement de la jonction ciment-formation (troisième interface). Formulée comme une tâche de segmentation supervisée, cette étude vise à concevoir un pipeline robuste, évalué par l'Intersection over Union (IoU), pour automatiser la qualification de la cimentation.

1.2 Outils utilisés

La gestion du projet est assurée via GitHub (dépôt) sous forme d'une architecture modulaire python. L'implémentation privilégie la reproductibilité, mais par respect des consignes, l'intégration du code source a été fait en annexe du notebook. Nous préconisons quand même de regarder plutôt le dépôt qui sera plus lisible. Les soumissions au challenge sont effectuées sous l'identifiant *Victor.camps2026*. Par transparence méthodologique, nous signalons l'usage d'assistants IA (Claude Haiku, Gemini-3) pour le codage et la révision de ce rapport.

1.3 Métrique de performance

La métrique utilisée pour évaluer les modèles dans ce Datachallenge est l'**Intersection Over Union** (IoU). L'IoU est définie mathématiquement comme :

$$\text{IoU} = \frac{\#X \cap Y}{\#X \cup Y}$$

où X est l'ensemble des pixels prédits avec une classe donnée et Y l'ensemble des pixels avec cette classe dans la Ground Truth.

2 Exploration des données

2.1 Format des données

Le jeu de données d'apprentissage est constitué de **4410 patches** provenant de 5 puits distincts (puits 2, 3, 4, 5 et 6). Les données brutes, initialement vectorisées et présentant une hétérogénéité de résolution (160×272 ou 160×160), sont annotées selon trois classes : **Background** (0), **Casing** (1) et **TIE** (2).

2.2 Distribution et déséquilibre des données

L'analyse exploratoire révèle une double asymétrie structurelle :

1. **Disparité géographique** : La répartition des échantillons par puits varie d'un facteur 1 à 5 (300 à 1600 patches), induisant un risque de sur-représentation des formations géologiques dominantes.

2. **Déséquilibre de classes** : Les structures d'intérêt sont largement minoritaires (**Casing** : 9%, **TIE** : 6%) face au fond (**Background** : 85%). Ce ratio de 15 :1 expose le modèle au risque de convergence triviale vers la classe majoritaire.

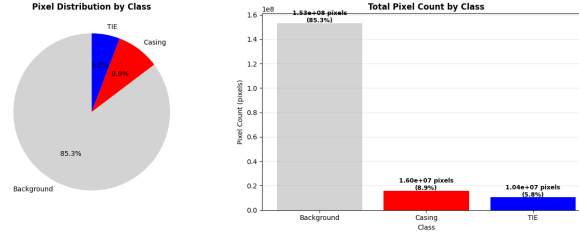


FIGURE 1 – Distribution des classes dans le dataset

Pour neutraliser ces biais, nous avons adopté une stratégie combinant pondération des classes (*class weights*) dans la fonction de coût, augmentation synthétique des données et expérimentation sur la fonction de perte.

2.3 Pipeline de prétraitement et justification des techniques

Le pipeline de prétraitement s'articule autour de trois opérations fondamentales garantissant la qualité des entrées du réseau de neurones :

1. **Standardisation dimensionnelle** : Les images sont redimensionnées uniformément à 160×160 pixels. Une stratégie d'interpolation différenciée est appliquée : bilinéaire (**INTER_LINEAR**) pour les images d'intensité afin de préserver la continuité du signal, et au plus proche voisin (**INTER_NEAREST**) pour les masques de segmentation afin de garantir l'intégrité des classes discrètes.
2. **Intégrité numérique** : Les éventuelles valeurs défectueuses (NaN) générées par les transformations géométriques sont détectées et imputées par 0 (signal nul).
3. **Normalisation radiométrique** : L'intensité des pixels est normalisée linéairement dans l'intervalle $[0, 1]$ selon $\tilde{x} = \frac{x - x_{min}}{x_{max} - x_{min}}$. Cette étape est indispensable pour homogénéiser la dynamique des entrées et faciliter la convergence de l'optimiseur.

Voici un exemple illustré :

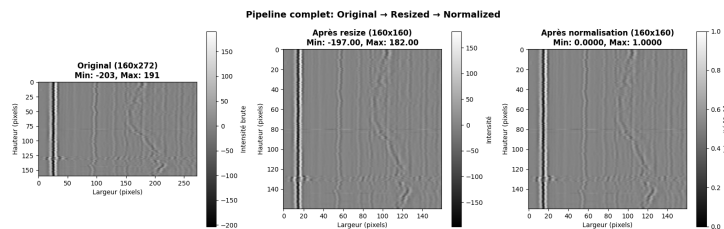


FIGURE 2 – Pipeline complet de prétraitement : Original → Redimensionné → Normalisé

3 Construction du modèle de référence

3.1 Motivations

Le choix de l'architecture U-Net s'appuie sur son efficacité démontrée pour la segmentation d'images biomédicales à faible contraste, une problématique analogue à l'analyse ultrasonore de puits. Sa structure encodeur-décodeur permet de concilier capture du contexte sémantique et localisation spatiale précise. Nos développements ultérieurs (Attention U-Net, Transfer Learning) s'inspirent de l'état de l'art, notamment des travaux sur l'Attention Res-UNet, qui valident l'apport des mécanismes d'attention pour le raffinement des cartes de caractéristiques dans les tâches de segmentation complexe.

3.2 Architecture générale

3.2.1 Structure du projet

Le projet adopte une architecture modulaire organisée en couches fonctionnelles distinctes, chacune responsable d'une étape du pipeline de segmentation. Cette organisation garantit la maintenabilité et permet de substituer facilement des composants sans affecter le reste du système.

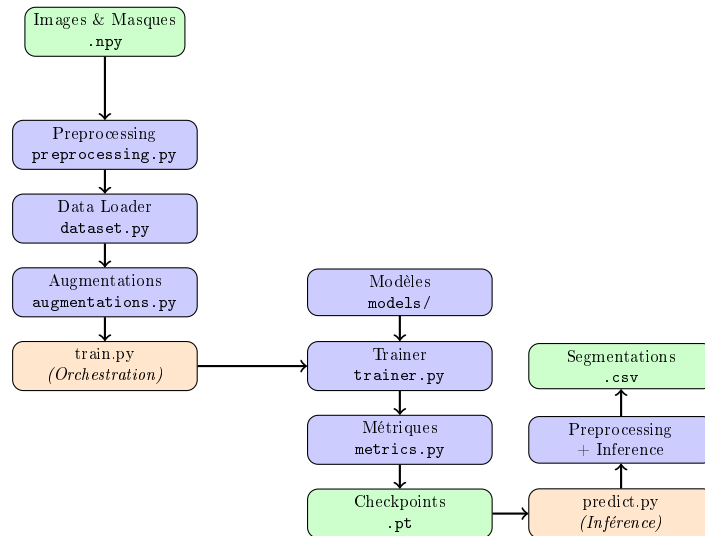


FIGURE 3 – Pipeline complet : entraînement (gauche) et inférence (droite)

Le schéma Figure 3 illustre l'architecture modulaire du projet en deux phases distinctes :

Phase d'entraînement : Le pipeline orchestre l'ingestion des données brutes, leur prétraitement (normalisation, interpolation) et leur augmentation dynamique. Les *DataLoaders* stratifiés alimentent la boucle d'apprentissage (**trainer.py**) qui gère la rétropropagation et l'évaluation continue des métriques. Un mécanisme d'*Early Stopping* garantit la sauvegarde du modèle optimal.

Phase d'inférence : Le script **predict.py** exploite le meilleur checkpoint pour prédire les masques sur les données de test, en garantissant la cohérence des prétraitements avant l'export

standardisé des résultats.

3.2.2 Description des modules

- **preprocessing.py** : Normalisation min-max, interpolation des images manquantes, conversion des masques en format compatible PyTorch.
- **dataset.py** : Chargement des données, création des DataLoaders PyTorch, gestion des splits train/validation avec stratification par puits.
- **augmentations.py** : Implémentation des transformations géométriques (rotations, flips) et du bruit Gaussien appliqués dynamiquement en entraînement.
- **models/** : Définitions des architectures UNet, UNet avec encodeur pré-entraîné, et variantes avec attention mechanisms.
- **trainer.py** : Boucle d'entraînement complète incluant forward pass, calcul de loss, backward propagation, et sauvegarde du meilleur modèle via Early Stopping.
- **metrics.py** : Calcul de l'Intersection over Union (IoU) pour chaque classe et accumulation des métriques moyennes via la classe **MetricsTracker**. Assure le suivi continu des performances pendant l'entraînement et la validation.
- **train.py** : Point d'entrée principal orchestrant l'entraînement avec arguments en ligne de commande pour configurer le modèle, le dataset et les hyperparamètres.
- **predict.py** : Interface d'inférence permettant de générer des prédictions segmentation sur de nouvelles images.

3.2.3 Modèle UNet Baseline

Le modèle UNet *baseline* constitue la référence de ce travail. Son architecture suit la structure encodeur-décodeur classique, adaptée à notre contexte d'images ultrasonores monocanal (160×160 pixels).

Architecture générale :

- **Encodeur** : 4 niveaux de convolution descendante avec max pooling
- **Goulot d'étranglement (Bottleneck)** : Bloc de convolution au niveau le plus profond
- **Décodeur** : 4 niveaux de déconvolution ascendante avec skip connections
- **Skip connections** : Concaténation des activations de l'encodeur aux niveaux correspondants du décodeur, préservant l'information spatiale

Blocs constitutifs :

Chaque niveau de l'encodeur et du décodeur utilise un **ConvBlock** composé de :

$$\boxed{\text{Conv2d}(k = 3) \rightarrow \text{BatchNorm} \rightarrow \text{ReLU}} \quad (\text{appliqué 2 fois})$$

Cette structure double-convolution avec normalisation par batch stabilise l'apprentissage et améliore la convergence. Le padding de 1 pixel préserve les dimensions spatiales au sein de chaque bloc.

Ce modèle baseline sert de point de comparaison pour les architectures ultérieures (avec Dropout, avec Focal Loss, avec transfer learning).

3.2.4 Modifications ultérieures

Ajout de Dropout : L'intégration de couches de *Dropout* ($p = 0.3$ après activation) a été évaluée comme stratégie de régularisation explicite pour prévenir le surapprentissage. Bien que théoriquement pertinente pour limiter la co-adaptation des neurones sur un dataset restreint, cette approche a empiriquement dégradé les performances par rapport au modèle de référence. Cette contre-performance indique que la perturbation stochastique interne interfère avec l'apprentissage des structures spatiales cohérentes des images ultrasonores. Il apparaît que les régularisations existantes (augmentation de données, pondération des classes) assuraient déjà un contrôle efficace du surapprentissage, rendant l'ajout de bruit supplémentaire contre-productif pour la convergence du modèle.

Ajout de bruit Gaussien dans l'augmentation : Nous avons enrichi le pipeline d'augmentation par l'injection dynamique de bruit gaussien ($\sigma = 0.02, p = 30\%$) afin de simuler les artéfacts d'acquisition ultrasonores. L'objectif est d'accroître la robustesse du modèle face aux variations de signal sans altérer la sémantique structurelle des interfaces.

Les résultats montrent une efficacité contrastée selon l'architecture : alors que le modèle *baseline* voit ses performances chuter face à ce bruit additionnel, les variantes pré-entraînées en bénéficient nettement. Ces dernières, grâce aux représentations robustes de leurs encodeurs (ResNet), exploitent efficacement ce bruit comme régularisateur, confirmant que la stratégie d'augmentation doit être calibrée en fonction de la capacité d'abstraction du réseau.

Exploration de fonctions de perte alternatives : Au-delà de la *CrossEntropyLoss* standard, nous avons évalué l'impact de critères d'optimisation spécifiques à la segmentation déséquilibrée :

- **Focal Loss** ($\gamma = 2.0$) : Conçue pour focaliser l'apprentissage sur les exemples difficiles ("hard negatives") afin de compenser le déséquilibre de classes (TIE $\approx 6\%$, Casing $\approx 9\%$).
- **Dice Loss** : Vise l'optimisation directe de la métrique IoU en maximisant le recouvrement spatial, favorisant ainsi la cohérence géométrique des prédictions.
- **Combined Loss** ($\mathcal{L}_{CE} + 0.5 \cdot \mathcal{L}_{Dice}$) : Approche hybride combinant la robustesse probabiliste de l'entropie croisée et la précision surfacique de la Dice Loss.

Les résultats indiquent que la *Focal Loss* n'offre pas de gain significatif face à une *Weighted CrossEntropy*, la pondération des classes suffisant apparemment à gérer le déséquilibre. En revanche, l'usage de la *Dice Loss* s'est avéré déterminant pour les performances finales des modèles avancés.

Intégration d'Attention Gates : Afin d'optimiser le transfert d'information via les *skip connections*, nous avons intégré des modules d'*Attention Gates* (AG). Contrairement à la concaténation standard qui fusionne toutes les caractéristiques spatiales sans distinction, les AG filtrent le signal de l'encodeur ($\mathbf{x}$) en se basant sur le contexte sémantique fourni par le décodeur ($\mathbf{g}$). Le mécanisme $\hat{\mathbf{x}} = \sigma(\psi(\mathbf{x}, \mathbf{g})) \odot \mathbf{x}$ génère une carte de saillance qui pondère les activations, amplifiant les zones pertinentes tout en atténuant le bruit de fond.

Cette modification architecturale s'est révélée bénéfique, surpassant les performances du modèle de base. En agissant comme un mécanisme de sélection adaptative, l'attention permet au

réseau de focaliser ses ressources sur les régions d'intérêt (interfaces fines), améliorant la précision de délimitation conformément à l'état de l'art en segmentation d'images.

Utilisation d'encodeurs pré-entraînés (Transfer Learning) : Pour pallier la limitation du volume de données, nous avons substitué l'encodeur standard par des architectures *ResNet* pré-entraînés sur ImageNet. Cette approche de *transfer learning* exploite la hiérarchie de caractéristiques visuelles déjà acquises : les couches superficielles détectent les primitives universelles (textures, contours) tandis que les couches profondes encodent des abstractions sémantiques robustes. Outre l'accélération de la convergence, cette initialisation réduit drastiquement le risque de surapprentissage inhérent aux petits datasets.

La stratégie d'implémentation repose sur un protocole en deux phases :

1. **Initialisation du décodeur** : L'encodeur est gelé (*frozen*) pour forcer l'adaptation exclusive du décodeur aux représentations fixes, prévenant la dégradation des poids pré-appris par des gradients instables ($LR : 1 \cdot 10^{-3}$).
2. **Raffinement global** : libération de l'encodeur pour un *fine-tuning* complet à très faible taux d'apprentissage ($1 \cdot 10^{-4}$), permettant une adaptation fine aux spécificités de l'imagerie ultrasonore sans oubli catastrophique.

Par souci de temps, nous n'avons testé que ResNet34, qui semblait être le bon compromis entre capacité et temps.

***Disclaimer** : je me suis rendu compte que pour la moitié des réseaux préentraînés, on n'a pas utilisé la phase 1 de l'entraînement !! (problème d'écriture de ligne de commande). On verra que ça n'a pas beaucoup d'incidence néanmoins.

4 Résultats

4.1 Métriques principales

Voici un tableau qui récapitule les données récoltées durant les apprentissages :

Modèle	Best IoU	Best Loss	Ep.	Score
Baseline	0.812	0.093	21	0.526
Dropout	0.766	0.135	16	0.415
Data Augm.	0.744	0.134	22	0.443
Focal Loss	0.810	0.008	32	0.508
Attention	0.844	0.083	69	0.574
Pretrained (Pt)	0.862	0.085	50	0.591
Pt + Gauss. Noise	0.835	0.084	22	0.583
Pt + Agg. Trans.	0.846	0.086	25	0.588
Pt + Comb. Loss	0.852	0.136	17	0.617
Pt + Dice Loss	0.885	0.065	15	0.648

TABLE 1 – Comparaison des métriques clés

Les modèles préentraînés dominent largement le tableau en ce qui concerne les scores du datachallenge, mais plus particulièrement le modèle préentraîné avec la loss dice, qui converge le plus vite, avec la meilleure IOU, et donc le meilleur score au datachallenge.

4.2 Evolution des différentes métriques

Voici l'ensemble des courbes tracées pour tous les modèles.

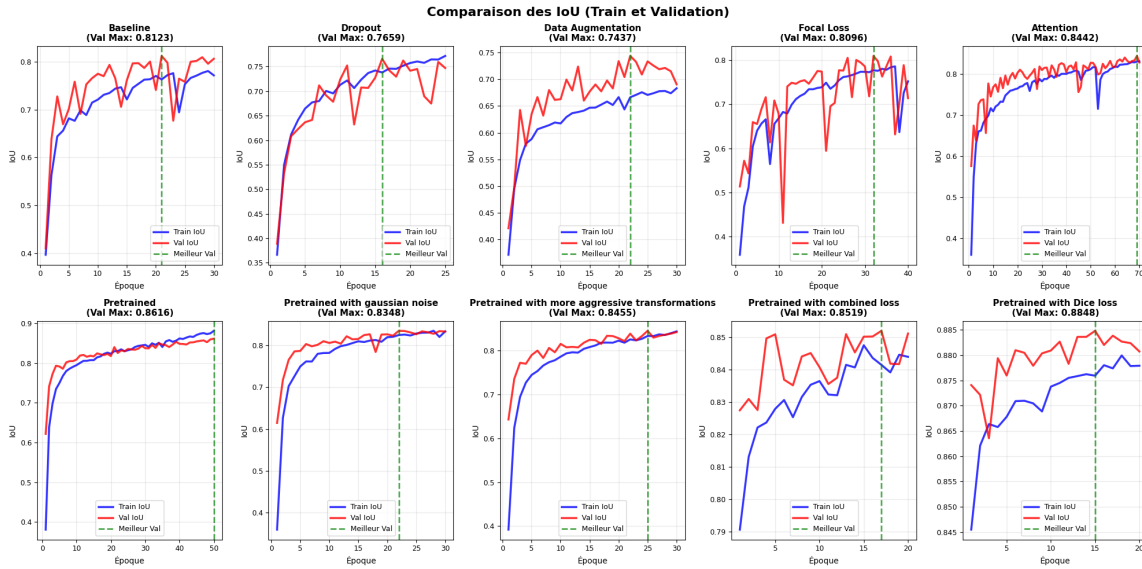


FIGURE 4 – Comparaison des IoU entre tous les modèles

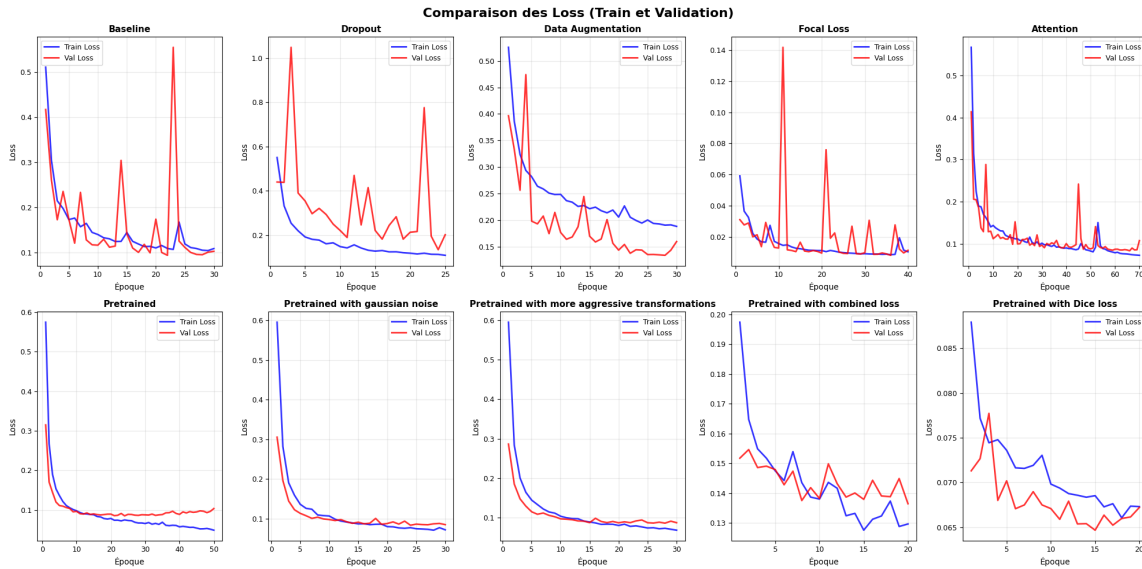


FIGURE 5 – Comparaison des Loss entre tous les modèles

L'analyse des profils de convergence révèle une progression logarithmique pour la majorité des modèles, sauf les modèles préentraînés. Une exception notable concerne les trois premières variantes pré-entraînées qui, en raison d'un dysfonctionnement dans la procédure de gel des poids (*freezing*), présentent une dynamique d'apprentissage similaire aux modèles standards, omettant la première phase de stabilisation.

Néanmoins, cette déviation procédurale n'a pas altéré la performance finale (IoU de 0.81 à 0.83), compensée par la robustesse des représentations des encodeurs ResNet et la stabilité offerte

par un faible taux d'apprentissage. Nous considérons que le manque à gagner est négligeable : l'analyse des modèles ayant suivi le protocole complet montre que la phase de raffinement (*fine-tuning*) n'apporte qu'un gain marginal d'environ 0.01 point d'IOU par rapport au plateau de 0.82 atteint lors de la phase initiale. Les résultats obtenus suggèrent donc que l'asymptote de performance de l'architecture a été atteinte, indépendamment de l'initialisation progressive.

4.3 Comparatif des métriques entre modèles

Pour avoir une vue d'ensemble des convergences de tous les modèles :

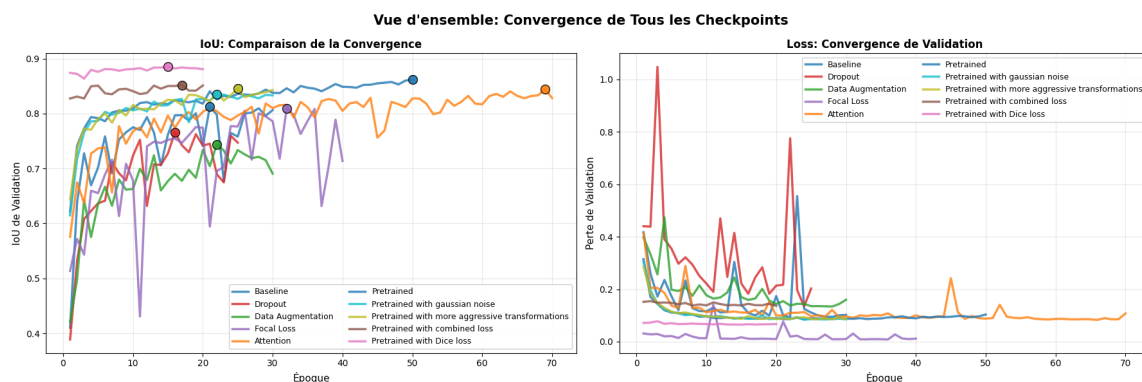


FIGURE 6 – Comparatif des métriques entre tous les modèles

Ici encore, on voit bien comment le modèle préentraîné avec la dice loss surnage en termes d'IOU. La loss est biaisée puisqu'on ne prend pas toujours la même fonction de perte, et donc on n'a pas la même échelle à prendre en compte.

5 Conclusion

5.1 Résumé des travaux réalisés

Ce projet de recherche s'est attaché à l'élaboration d'une solution algorithmique de segmentation sémantique appliquée à l'imagerie ultrasonore, dans le contexte spécifique de l'analyse de puits de forage. L'enjeu principal résidait dans l'identification précise de trois structures géologiques et mécaniques distinctes (Ciment, Casing, TIE) au sein d'un jeu de données restreint et présentant un fort déséquilibre de classes.

Notre méthodologie a reposé sur une exploration systématique de différentes stratégies de modélisation :

- **Architecture de référence** : Implémentation d'un réseau U-Net standard à initialisation aléatoire (31M paramètres).
- **Variantes architecturales** : Intégration de mécanismes de régularisation (Dropout) et d'attention (Attention Gates).
- **Enrichissement des données** : Application de techniques d'augmentation (transformations géométriques, injection de bruit gaussien).
- **Fonctions de coût** : Évaluation comparative de critères d'optimisation (Focal Loss, Dice Loss, Combined Loss).

- **Apprentissage par transfert** : Exploitation d'encodeurs pré-entraînés (famille ResNet).
- **Stratégies d'entraînement** : Mise en œuvre de protocoles de *fine-tuning* progressif.

5.2 Conclusion et résultats

Ce travail de recherche a permis d'optimiser une architecture de segmentation pour l'imagerie de puits en environnement contraint (données limitées, déséquilibre de classes). L'analyse comparative consacre le modèle U-Net avec encodeur ResNet-34 pré-entraîné et fonction de coût *Dice Loss* comme solution optimale, validant la supériorité du transfert d'apprentissage pour cette tâche.

Avec un protocole d'entraînement séquentiel et des augmentations par bruit gaussien, cette configuration atteint une **IoU de 0.885** sur l'ensemble de validation (+8.95% par rapport à la baseline). Ce résultat confirme qu'un modèle à forte capacité de représentation, correctement régularisé, offre le meilleur compromis performance-généralisation.

Finalement, ces performances se traduisent par un classement de 15^e/52 au niveau global (4^e de la promotion), soulignant la validité d'une démarche méthodique combinant connaissances métiers et stratégies avancées de Deep Learning.